

Cover Page

â€¢ **Course:** SE4030 â€” Secure Software Development
â€¢ **Assignment:** Vulnerability Assessment, Remediation, and OAuth Integration
â€¢ **Application:** Car Booking Web Application
â€¢ **Team Members:** _Add member names and index numbers here_
â€¢ **Submission Date:** _Add date_

1. Introduction

The selected application is a PHP/MySQL-based car booking portal that enables customers to browse vehicles, make reservations, and allows administrators to manage the fleet. The legacy codebase relied on ad-hoc scripts with minimal security controls, resulting in multiple critical vulnerabilities. This project modernised the security posture, remediated the highest-risk flaws, and introduced Google OpenID Connect login to strengthen authentication.

2. Assessment Methodology

The team adopted a repeatable process that combined manual inspection with automated assistance:

- 1. **Source code review:** Analysed PHP, HTML, and SQL files to identify insecure patterns such as unsanitised input, outdated cryptography, and missing authorisation checks.
- 2. **Dependency and configuration audit:** Reviewed configuration files and database schema to uncover hard-coded secrets and weak defaults.
- 3. **Manual functional testing:** Exercised login, registration, booking, and administrative workflows in a local environment to validate exploitability and confirm fixes.
- 4. **Automated linting:** Executed 'php -l' against modified files to ensure syntax correctness and reduce regression risk.
- 5. **Threat modelling:** Mapped attack surfaces (public forms, file uploads, administrative actions, OAuth callbacks) to prioritise remediation effort.

3. Vulnerability Summary

| Vulnerability | Location | Risk | Status

--- | ----- | ----- | ----- | -----

- 1 | Hard-coded root database credentials and lack of connection error handling | 'inc/connection.inc.php', 'admin/includes/connection.inc.php' | High | Fixed
- 2 | Session cookies missing secure attributes, enabling session hijacking | Global

session bootstrap | Medium | Fixed

3 | SQL injection in customer login flow | 'login.php' | Critical | Fixed

4 | Weak MD5 password hashing and plaintext comparison | 'login.php', 'register.php' | High | Fixed

5 | Stored cross-site scripting via booking messages and vehicle metadata | 'carDetails.php', 'admin/manage-bookings.php', 'my_account.php' | High | Fixed

6 | Cross-site request forgery on booking, registration, and admin actions | Multiple forms ('register.php', 'login.php', 'carDetails.php', admin modules) | High | Fixed

7 | Arbitrary file upload leading to potential remote code execution | 'admin/post-avehical.php' | Critical | Fixed

4. Detailed Findings and Fixes

4.1 Hard-coded database credentials and missing error handling

â€¢ **Original issue:** The legacy connection scripts embedded the 'root' MySQL user with a blank password and did not check for connection failures. Any compromise of the repository or webroot exposed production credentials. Additionally, the application silently proceeded when the database connection failed, leading to unpredictable behaviour.

â€¢ **Impact:** Attackers could reuse the leaked root credentials to access the production database remotely. Local attackers could modify the file to pivot into the database or suppress error messages to mask malicious activity.

â€¢ **Fix:** Replaced direct credentials with an environment/configuration bootstrap ('inc/config.php' or environment variables), enforced strict MySQLi error reporting, and failed closed with a 500 response on connection errors.

4.2 Insecure session cookies

â€¢ **Original issue:** Sessions were started with default PHP settings, which omit the 'Secure', 'HttpOnly', and 'SameSite' flags. On HTTPS deployments, cookies could be transmitted over HTTP or accessed via JavaScript.

â€¢ **Impact:** Session identifiers were exposed to network attackers or could be stolen through XSS, allowing account takeover.

â€¢ **Fix:** Added a central 'security.inc.php' bootstrap that sets secure session cookie parameters before 'session_start()', ensuring cookies are HTTPS-only (when applicable), 'HttpOnly', and use 'SameSite=Lax' to resist CSRF.

4.3 SQL injection in login workflow

â€¢ **Original issue:** The login handler interpolated raw '\$_POST' values into SQL queries without sanitisation or parameterisation: 'SELECT * FROM users WHERE email='\$username' AND password='\$password''. Attackers could authenticate without valid credentials by injecting SQL payloads.

â€¢ **Impact:** Complete bypass of authentication and exposure of all user data.

â€¢ **Fix:** Rebuilt the login logic using prepared statements, strict input validation, and limited column selection. The query now binds parameters and verifies hashed passwords server-side.

4.4 Weak password hashing

â€¢ **Original issue:** User passwords were hashed with 'md5()' during registration and compared with MD5 digests on login. MD5 is collision-prone and GPU-crackable within seconds.

â€¢ **Impact:** A database leak would immediately expose all user passwords, enabling credential stuffing.

â€¢ **Fix:** Migrated to PHP's 'password_hash()'/ 'password_verify()' APIs with the default bcrypt algorithm, regenerated password hashes for default accounts, and updated the schema to accommodate stronger hashes.

4.5 Stored cross-site scripting (XSS)

â€¢ **Original issue:** User-supplied values such as booking messages, vehicle titles, and other metadata were stored unfiltered and echoed directly into HTML contexts throughout the site (customer dashboards and admin panels).

â€¢ **Impact:** Authenticated attackers could inject JavaScript that executes in other users' browsers, leading to session theft or administrative compromise.

â€¢ **Fix:** Introduced sanitisation helpers for inbound data, escaped all dynamic output with 'htmlspecialchars()', and audited templates to ensure untrusted values are never rendered raw.

4.6 Cross-site request forgery (CSRF)

â€¢ **Original issue:** High-impact state-changing forms (login, registration, bookings, admin approvals, vehicle uploads) lacked CSRF tokens. Browsers would happily submit forged requests triggered from malicious sites.

â€¢ **Impact:** Attackers could trick authenticated users into confirming bookings, adding vehicles, or changing account data without consent.

â€¢ **Fix:** Implemented reusable CSRF helpers ('generate_csrf_token', 'require_valid_csrf_token') and embedded hidden tokens in every sensitive form. Server-side handlers now reject requests with missing or invalid tokens.

4.7 Arbitrary file upload

â€¢ **Original issue:** The admin vehicle upload feature accepted any file name and extension, storing it directly under the public 'img/vehicleimages/' directory without validation or randomisation.

â€¢ **Impact:** An attacker with admin access could upload a PHP web shell and achieve remote code execution. Even non-admin attackers could exploit stolen admin sessions (see CSRF issue) to plant malware.

â€¢ **Fix:** Restricted uploads to approved MIME types, generated randomised filenames, stored metadata safely, and verified move operations before persisting references.

5. OAuth / OpenID Connect Integration

A new Google OpenID Connect login flow was added to support social sign-on. The implementation:

- â€¢ Registers the application with Google Cloud and stores the client credentials securely in 'inc/config.php' or environment variables.

- â€¢ Uses an explicit login endpoint ('oauth/google_login.php') that builds the Google authorisation URL with cryptographically strong 'state' and 'nonce' parameters.

- â€¢ Handles the callback ('oauth/google_callback.php') by validating the 'state', exchanging the authorisation code for tokens via cURL, verifying the ID token signature and claims, and provisioning or linking local accounts.

- â€¢ Integrates with the existing session management and CSRF framework to ensure a consistent security posture.

6. Residual Risks and Deferred Items

- â€¢ **Transport security:** Enforcing HTTPS and HSTS requires web server configuration that is outside the application repository. Deployment guides should mandate TLS.

- â€¢ **Rate limiting:** Brute-force protection for login and OAuth callbacks was not implemented due to time constraints. Future iterations should add IP-based throttling or CAPTCHA challenges.

- â€¢ **Comprehensive logging:** The remediation focused on security-critical functionality; centralised auditing and monitoring were not added but remain a recommended control.

7. Best Practice Recommendations

- â€¢ Incorporate secure code review and threat modelling into every release cycle.

- â€¢ Store secrets outside source control (environment variables or secret managers) and rotate them regularly.

- â€¢ Enforce automated static analysis (e.g., PHPStan, Psalm) and dependency scanning in CI pipelines.

- â€¢ Adopt a defence-in-depth mindset: layered controls (input validation, output encoding, authorisation, logging) help contain the impact of any single flaw.

- â€¢ Keep third-party SDKs and libraries updated, and monitor for advisories affecting OAuth integrations.

8. Testing Evidence

- â€¢ PHP syntax checks were executed for all modified PHP files using 'php -l'.

- â€¢ Manual regression testing confirmed that login, registration, booking, admin workflows, and Google OAuth login operate as expected with the new security controls.

9. References

- â€¢ OWASP Top Ten 2021

- â€¢ Google Identity Platform documentation

- â€¢ PHP Manual: Sessions and Password Hashing API